TECHNICAL REPORT

APT Memory Forensics Pipeline

A Velociraptor-Based Feature Engineering Framework for Advanced Persistent Threat Detection

Version 1.0 | 2025

SASTRA Deemed University — APT Research Group

Abstract

Advanced Persistent Threats (APTs) represent a sophisticated class of cyber intrusions characterised by prolonged dwell times, lateral movement, and deliberate evasion of conventional signature-based defences. This report describes the APT Memory Forensics Pipeline, a modular, open-source framework that transforms raw Velociraptor NDJSON memory artefacts into a structured, machine-learning-ready feature matrix for APT detection research.

The pipeline ingests ten Velociraptor artefact types covering process lists, memory regions, loaded DLLs, kernel handles, mutex objects, thread stacks, virtual address descriptors, network connections, process environment blocks, and token impersonation events, and distils them into 35 decisional binary flags and 14 conditional numeric features per process. An APT risk score and two labels (ground-truth and heuristic) are produced alongside identity metadata. Companion scripts automate multi-dump dataset assembly and training of Logistic Regression, Random Forest, and XGBoost classifiers under five-fold stratified cross-validation.

1. Introduction

Memory forensics has emerged as one of the most reliable techniques for detecting sophisticated malware that deliberately avoids touching disk. Adversaries employing process hollowing, DLL injection, reflective loading, and token impersonation leave artefacts almost exclusively in volatile memory. Existing tooling either produces human-readable reports unsuitable for automated analysis, or requires deep expertise to interpret raw dumps.

Velociraptor is an open-source digital forensics and incident response (DFIR) platform that can collect structured artefacts from live endpoints at scale. Its output, however, is a collection of NDJSON files that must be parsed, joined, and transformed before any machine learning model can be applied.

The APT Memory Forensics Pipeline addresses this gap. It provides a reproducible, auditable Python pipeline that:

- Parses and joins all ten relevant Velociraptor artefact types.
- Engineers 49 features (35 decisional + 14 conditional) grounded in real APT tradecraft.
- Produces a per-process CSV dataset with identity metadata, features, a weighted risk score, and dual labels.
- Supports multi-dump dataset assembly via a combiner script.
- Provides a training script for three classifiers with imbalance handling, cross-validation, and evaluation plots.

2. Background and Related Work

2.1 APT Threat Model

APTs are nation-state or organised-crime threat actors that conduct targeted, multi-stage intrusions. The MITRE ATT&CK framework catalogues over 200 techniques used across reconnaissance, initial access, execution, persistence, privilege escalation, defence evasion, credential access, lateral movement, and exfiltration. The pipeline specifically targets the following ATT&CK technique clusters:

ATT&CK Technique	ID	Pipeline Coverage
Process Injection	T1055	VAD, thread, and DLL features
Process Hollowing	T1055.012	hollow_process_indicator
Masquerading	T1036	17 masquerade detection rules
Token Impersonation	T1134	Impersonation artefact features
C2 Communication	T1071	Network features and C2 port list
Suspicious Mutexes	T1480	Mutant handle scanning

2.2 Memory Forensics with Velociraptor

Velociraptor executes Velocidex Query Language (VQL) artefacts on live endpoints and streams results as NDJSON. Each artefact exposes a specific subsystem: Windows.System.Pslist queries the process list via the Windows API; Windows.Memory.ProcessInfo reads the Process Environment Block (PEB); Windows.System.VAD enumerates Virtual Address Descriptor nodes from kernel memory. The pipeline ingests all ten artefacts and joins them on process ID.

3. System Architecture

3.1 Component Overview

Module	Script	Responsibility
Feature Pipeline	apt_baseline_pipeline.py	Load artefacts, engineer all features, produce per-dump CSV
Combiner	apt_combiner.py	Iterate over dump folders, call pipeline, assemble master dataset
Trainer	apt_train.py	Load master CSV, train classifiers, evaluate, save plots and reports

3.2 Data Flow

The end-to-end data flow proceeds through nine stages:

1. NDJSON artefact ingestion — ten files loaded line-by-line with json.loads().
2. Pslist normalisation — column aliasing, type coercion, parent-name resolution.
3. Process-level feature engineering — entropy, Levenshtein distance, path validity, user checks.
4. PEB mismatch detection — ProcessInfo ImagePathName compared against Pslist Exe path.
5. Masquerade detection — seventeen rule-based flags applied per process.
6. Artefact-level features — DLL, handle, mutant, thread, VAD features merged by PID.
7. Network and impersonation features — connection analysis, C2 detection, token inspection.
8. Composite features — hollow process indicator, injection-with-C2 derived flags.
9. Scoring and labelling — weighted APT risk score, heuristic label, ground-truth label.

3.3 Input Artefacts

Artefact File	PID Column	Key Fields Used
Windows.System.Pslist.json	Pid	Name, Exe, PPid, CommandLine, Username, TokenIsElevated
Windows.Memory.ProcessInfo.json	Pid	ImagePathName (PEB path for mismatch detection)
Windows.System.DLLs.json	Pid	ModuleName, ModulePath
Windows.System.Handles.json	ProcPid	Name (high-privilege target matching)
Windows.Detection.Mutants/ Handles.json	ProcPid	Type, Name (per-process suspicious mutex count)
Windows.Detection.Mutants/ ObjectTree.json	None (dump-level)	Type, Name (global suspicious mutex count)
Windows.System.Threads.json	Pid	Filename (blank = anonymous memory thread)
Windows.System.VAD.json	Pid	Type, ProtectionMsg, MappingName
Windows.Network.Netstat.json	Pid	Raddr.IP, Raddr.Port, Laddr.Port, Status
Windows.Detection.Impersonation.json	ProcPid	ImpersonationToken, Username

4. Feature Engineering

4.1 Feature Taxonomy

All features are derived from first principles of Windows process internals and known APT tradecraft. They are divided into two categories: decisional features (binary flags designed for direct interpretability by analysts) and conditional features (numeric context values that provide quantitative signal to classifiers).

4.2 Decisional Features (Binary Flags)

Feature	Source	Definition
typosquat_flag	Pslist	Levenshtein distance 1–2 from a known-good Windows process name
username_mismatch	Pslist	Critical system process running under a non-system account
multi_instance_flag	Pslist	Singleton process (lsass, services, wininit, smss, lsm) has >1 instance
path_invalid_flag	Pslist	Critical process not running from its expected filesystem path
proc_peb_mismatch	ProcessInfo	Pslist Exe path differs from PEB ImagePathName (process hollowing indicator)
masquerade_score	Pslist	Sum of all 17 individual masquerade rule flags (range 0–17)
boot_order_violation	Pslist	smss PID ≥ csrss PID or csrss PID ≥ wininit PID
parent_also_suspicious	Pslist	Parent process has masquerade_score > 0
dll_path_anomaly	DLLs	Any loaded DLL path contains \Temp\, \AppData\, \Downloads\, or \Desktop\
dll_outside_sys32	DLLs	Any DLL loaded outside System32, SysWOW64, or Program Files
handle_high_privilege_target_count	Handles	Count of handles targeting lsass, winlogon, csrss, or services
mutant_suspicious_name_count	Mut/Hdl	Per-process count of mutex names matching known-malicious patterns
global_susp_mutex_count	Mut/Tree	Dump-level count of suspicious mutex names in the object tree
anon_memory_thread_count	Threads	Threads with blank Filename, indicating execution from anonymous memory

vad_private_exec_region_count	VAD	VAD regions that are both Private and executable (shellcode staging indicator)
vad_region_with_no_file_backing_exec_count	VAD	Executable VAD regions with no mapped file (MappingName is blank)
c2_port_flag	Netstat	Connection to a known C2 port: {4444,5555,1337,6666,8080,8443,9001,3389,1080,9999,31337}
beacon_pattern	Netstat	Same remote IP contacted more than 5 times (beaconing indicator)
impersonation_detected	Imperson.	Process has an active impersonation token
high_privilege_impersonation	Imperson.	Impersonation token belongs to SYSTEM or Administrator
cross_user_impersonation	Imperson.	Process is impersonating a user different from its owner
hollow_process_indicator	DLLs+Net	dll_count < 5 AND external_conn_count > 0 (hollowed process calling home)
injection_with_c2	DLLs+Net	in_mem_dll_count > 0 AND c2_port_flag = 1 (injected payload with C2)

4.2.1 Masquerade Detection Rules (17 flags)

Flag	Condition Checked
masq_svchost	Path outside System32 OR parent is not services.exe
masq_lsass	Path outside System32 OR parent is not wininit.exe
masq_csrss	Parent is not smss.exe
masq_wininit	Parent is not smss.exe
masq_services	Parent is not wininit.exe
masq_smss	Parent PID is not 4 (System)
masq_winlogon	Parent is not smss.exe
masq_explorer	Path does not start with C:\Windows\explorer.exe
masq_taskhost	Path outside System32
masq_dllhost	Path outside System32 OR spawned by Office/browser process
masq_powershell	Spawned by Office/browser OR uses encoded/download/hidden flags
masq_cmd	Spawned by Office or browser process
masq_mshta	Parent is not explorer.exe
masq_regsvr32	Spawned by Office OR command contains scrobj/.sct/http
masq_rundll32	Command references suspicious path OR spawned by Office/browser

masq_wscript	Spawned by Office process
masq_certutil	Command contains -decode, -urlcache, or -f http

4.3 Conditional Features (Numeric)

Feature	Source	Description
name_entropy	Pslist	Shannon entropy of the process name characters (excluding .exe)
min_lev_distance	Pslist	Minimum Levenshtein distance to any known-good process name
instance_count	Pslist	Number of running instances of this process name
is_elevated	Pslist	TokenIsElevated flag from the process access token
child_count	Pslist	Number of direct child processes spawned by this process
dll_count	DLLs	Total number of DLLs loaded into the process address space
dll_name_entropy_avg	DLLs	Average Shannon entropy across all loaded DLL names
handle_count	Handles	Total number of kernel handles opened by the process
thread_count	Threads	Total number of threads belonging to the process
vad_exec_private_ratio	VAD	vad_private_exec_region_count divided by total VAD region count
external_conn_count	Netstat	Connections to non-RFC-1918 / non-loopback IP addresses
unique_remote_ips	Netstat	Number of distinct remote IP addresses contacted
rare_listen_port	Netstat	Listening on port < 1024 excluding {80,443,135,139,445,53,22,21}
impersonation_count	Imperson.	Total count of impersonation events recorded for the process

5. APT Risk Score

Each process receives a weighted integer risk score aggregated from the most forensically significant decisional features. The weights penalise high-confidence indicators of compromise more heavily:

Feature	Weight	Rationale
injection_with_c2	x4	Strongest combined APT indicator
high_privilege_impersonation	x4	SYSTEM-level token theft
masquerade_score	x3	Per-rule masquerade evidence
typosquat_flag	x3	Name deception technique
username_mismatch	x3	Privilege anomaly
multi_instance_flag	x3	Singleton violation
path_invalid_flag	x3	Path deception technique
proc_peb_mismatch	x3	Process hollowing evidence
mutant_suspicious_name_count	x3	Known-bad mutex patterns
anon_memory_thread_count	x3	Shellcode thread injection
c2_port_flag	x3	Known C2 port contact
impersonation_detected	x3	Active token impersonation
cross_user_impersonation	x3	Cross-account impersonation
handle_high_privilege_target_count	x2	Handle-based privilege escalation
global_susp_mutex_count	x2	Dump-level malware mutex IOC
vad_private_exec_region_count	x2	Private executable memory
vad_region_with_no_file_backing_exec_count	x2	Unbacked executable memory
beacon_pattern	x2	Periodic C2 beaconing
hollow_process_indicator	x2	Hollowed process heuristic
parent_also_suspicious	x2	Suspicious process chain

6. Labelling Strategy

6.1 Ground-Truth Label (dump_label)

The dump_label column carries the analyst-assigned ground truth for the entire memory dump: 0 for benign, 1 for malicious, and -1 for unknown. All processes within a dump share the same label. This label is used as the training target and is excluded from features during model training.

6.2 Heuristic Label (heuristic_label)

A process receives heuristic_label = 1 if any of the following conditions fire:

- masquerade_score > 0
- typosquat_flag = 1
- username_mismatch = 1
- multi_instance_flag = 1
- path_invalid_flag = 1
- proc_peb_mismatch = 1
- c2_port_flag = 1
- impersonation_detected = 1
- high_privilege_impersonation = 1
- mutant_suspicious_name_count > 0
- anon_memory_thread_count > 0
- vad_region_with_no_file_backing_exec_count > 0

The heuristic label is useful for unsupervised and semi-supervised experimental setups but is excluded from supervised training features to avoid label leakage.

7. Output Dataset Schema

The output CSV follows this exact column order:

Group	Columns
Identity (9)	dump_id, PID, ppid, process_name, parent_name, Path, CommandLine, Username, path_validity_label
Decisional (35)	typosquat_flag, username_mismatch, multi_instance_flag, path_invalid_flag, proc_peb_mismatch, masquerade_score, masq_svchost … masq_certutil [17 flags], boot_order_violation, parent_also_suspicious, dll_path_anomaly, dll_outside_sys32, handle_high_privilege_target_count, mutant_suspicious_name_count, global_susp_mutex_count, anon_memory_thread_count, vad_private_exec_region_count, vad_region_with_no_file_backing_exec_count, c2_port_flag, beacon_pattern, impersonation_detected, high_privilege_impersonation, cross_user_impersonation, hollow_process_indicator, injection_with_c2
Conditional (14)	name_entropy, min_lev_distance, instance_count, is_elevated, child_count, dll_count, dll_name_entropy_avg, handle_count, thread_count, vad_exec_private_ratio, external_conn_count, unique_remote_ips, rare_listen_port, impersonation_count

Score	apt_risk_score
Labels	dump_label, heuristic_label

8. Model Training Pipeline

8.1 Feature Selection for Training

All identity columns, string columns, apt_risk_score, and heuristic_label are excluded from the model feature matrix to prevent data leakage. Only rows with dump_label in {0, 1} are included in supervised training.

8.2 Class Imbalance Handling

Real-world memory dumps contain far more benign than malicious processes. The pipeline handles this imbalance in order of preference:

10. SMOTE (Synthetic Minority Oversampling Technique) applied per fold if imbalanced-learn is installed.
11. class_weight='balanced' used as a fallback for all classifiers if SMOTE is unavailable.

8.3 Classifiers

Classifier	Key Hyperparameters	Notes
Logistic Regression	max_iter=1000, solver=lbfgs	Linear baseline; coefficients used as feature importance
Random Forest	n_estimators=200	Non-linear ensemble; Gini impurity feature importance
XGBoost	n_estimators=200, max_depth=6, lr=0.1	Gradient boosting; optional xgboost package

8.4 Evaluation Protocol

- 5-fold stratified cross-validation preserving class ratio in every fold.
- Metrics reported: F1, Precision, Recall, ROC-AUC (mean and standard deviation across folds).
- Out-of-fold predictions aggregated for confusion matrix and ROC curve plots.
- Models retrained on full dataset after CV for feature importance extraction.

8.5 Outputs

Output File	Contents
classification_report.txt	Per-class precision, recall, F1 for each model on full refit
feature_importance.csv	Feature importances for all models combined
cv_metrics_summary.csv	Mean F1, Precision, Recall, ROC-AUC, F1-std per model

roc_curve_<Model>.png	OOF ROC curve with AUC annotation
confusion_matrix_<Model>.png	OOF confusion matrix heatmap
model_comparison.png	Side-by-side bar chart of all metric means across models
feature_importance_<Model>.png	Top-25 feature importance bar chart per model

9. Limitations and Future Work

9.1 Current Limitations

- Dump-level labelling: all processes in a dump share one label; process-level ground truth requires manual annotation.
- Static port list: C2 port detection relies on a hardcoded set of 11 well-known ports; adaptive adversaries may use arbitrary ports.
- No kernel integrity checks: rootkits exploiting DKOM may hide processes from Pslist entirely.
- Single-snapshot analysis: temporal correlation across multiple snapshots is not yet implemented.
- in_mem_dll_count defaults to zero: the artefact set does not expose per-DLL memory flags; injection_with_c2 will not fire without this signal.

9.2 Future Directions

- Process-level labelling via sandboxed execution and behavioural tagging.
- Graph-based features encoding parent-child and sibling process relationships.
- Temporal beaconing analysis across multiple sequential memory snapshots.
- Integration of Windows Event Log artefacts (4688, 4624, 4648) for cross-source correlation.
- MITRE ATT&CK tactic-level multi-label classification.

10. Dependencies

Package	Version	Purpose
Python	≥ 3.8	Runtime
pandas	≥ 1.3	Dataframe operations and CSV I/O
numpy	≥ 1.21	Numerical computations
scikit-learn	≥ 1.0	Classifiers, cross-validation, metrics
xgboost	≥ 1.6 (optional)	Gradient boosting classifier
imbalanced-learn	≥ 0.9 (optional)	SMOTE oversampling
matplotlib	≥ 3.4	Plot generation
seaborn	≥ 0.11	Confusion matrix heatmaps

tabulate	any (optional)	Formatted summary tables in combiner output

References

[1] MITRE Corporation. MITRE ATT&CK Framework. https://attack.mitre.org, 2024.

[2] Velocidex Enterprises. Velociraptor: Endpoint Visibility and Collection Tool. https://www.velocidex.com, 2024.

[3] Russinovich, M., Solomon, D., Ionescu, A. Windows Internals, 7th Edition. Microsoft Press, 2017.

[4] Chawla, N. et al. SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16:321-357, 2002.

[5] Chen, T., Guestrin, C. XGBoost: A Scalable Tree Boosting System. KDD, 2016.

[6] Malin, C., Casey, E., Aquilina, J. Malware Forensics Field Guide for Windows Systems. Syngress, 2012.

[7] Ligh, M. et al. The Art of Memory Forensics. Wiley, 2014.